

Security audit

RAG SaaS Platform

Security report

Security report — 2026-10-03

Verdict : PARTIEL, non certifié

Production non contactée. Aucun SQL, extension, migration ou policy appliqué sur staging. Le réseau vers la cible explicitement autorisée est inaccessible; le profil local n'a qu'un credential placeholder. Le secret partagé dans le chat doit être tourné, sans être recopié ici.

Preuves et limites

- Inventaire reproductible : 936 opérations HTTP, 177 tables ORM.

Les dépendances d'authentification sont inventoriées, mais le filtre tenant de chaque branche et chaque méthode n'est pas certifié.

- Tests A/B PostgreSQL et RLS sous rôle non-BYPASS préparés : 11 cas

skippés, zéro preuve live. Une réponse 404 d'une route absente ne constitue pas une preuve d'isolation.

- Deux défauts confirmés dans la phase corrective antérieure :

historique de message cross-conversation et détachement Stripe sans validation du customer. Correctifs et replays SQLite/mocks décrits dans [le rapport Phase 2](#). Pas de validation Stripe ou PostgreSQL live.

- Garde-fous staging : host/port/user/database exacts, SSL, refus des

overrides, dotenv sélectionné explicitement, clés locales éphémères, désactivation dotenv tiers prouvée par test, pas de provider hérité.

- RLS proposé uniquement pour tables directes tenant-scoped, rôle de

laboratoire restreint. Tables indirectes et contexte transactionnel de l'API/workers restent à traiter, sans appliquer une policy générique.

Revue spécialisée

La revue spécialisée read-only est toujours en cours à cette rédaction. Aucun résultat final du spécialiste n'est disponible : **ne pas inventer un nombre de vulnérabilités ni annoncer absence de failles**. Ce document sera complété à réception des findings vérifiés.

Gates non atteints

1. Staging PostgreSQL accessible et provisionné avec un nouveau secret local.

2. Migrations et catalogues vérifiés, RLS testée avec rôle runtime exact.
3. Contrôles owner et cross-tenant pour toutes méthodes/actions exposées.
4. Findings spécialisés réconciliés, reproduits et corrigés.
5. Dépendances locales compatibles et suite backend complète verte.
6. Backup/restore et rollback éprouvés sur base distincte.

Sources : [inventaire](#), [décision RLS](#), [staging](#), [tests](#).

Security architecture

Security Model

Modele de securite multi-tenant, RBAC, permissions tools et audit. Pour les composants, voir OVERVIEW.md.

Multi-tenancy

Principe : chaque table tenant-scoped porte un `org_id`, et la plupart appliquent RLS PostgreSQL en plus des checks applicatifs.

Double verification

1. Applicatif : `require_permission("resource:action")` sur chaque endpoint sensible.
2. DB : RLS (`org_id = current_setting`), activee sur les tables critiques

via migrations Alembic (`0087_enable_row_level_security_remaining_tables.py`).

Benefice : un bug applicatif ne peut pas exposer les donnees d une autre org -- la DB refuse.

Isolation cross-agent

- Long-term memory : cle (`agent_id`, `user_id`, `key`), cross-run mais isolee par agent.
- Short-term memory : `session_id` (string), pas de fuite cross-session.
- Tools : `ToolPermission` per-agent-per-tool.

RBAC granulaire

52 permissions organisees par ressource (`api/security/permission_catalog.py`).

Roles built-in

Role	Scope
owner	Tout, y compris billing + suppression org
admin	Tout sauf billing critique
member	Lecture + ecriture selon permissions explicites
viewer	Lecture seule

Resource permissions

En plus des roles, `ResourcePermission` permet des permissions par ressource individuelle.

Tool permissions (agents)

Chaque agent a des ToolPermission par tool :

- allow : le tool peut etre execute.
- deny : le tool est refuse (meme si selectionne par le LLM).

Verification : check_tool_permission(...) avant chaque execution de tool.

Auth

Sessions

- Cookie HttpOnly, Secure (si COOKIE_SECURE=true), SameSite=Lax.
- Session store cote serveur (Redis ou DB).
- Expiration configurable.

API keys

- Format sk_live_... / sk_test_...
- Hashees en DB (jamais en clair).
- Scope par organisation.
- Revocation immediate.

SSO / Enterprise

- OIDC / SAML via enterprise_sso.py.
- Mapping des groupes IdP -> roles org.

2FA / WebAuthn

- TOTP (two_factor.py).
- WebAuthn (cles physiques, biometrie).
- Recovery codes.

Audit

- AuditLog : chaque action sensible (login, permission change, billing, suppression).
- Champs : actor_id, action, resource_type, resource_id, metadata, ip, user_agent, timestamp.
- Retention configurable.

Chiffrement

Secrets en transit

- TLS partout (HTTPS only en prod).
- Webhooks : signature HMAC verifiee (Stripe, Paystack).

Secrets au repos

- STRIPE_SECRET_KEY, PAYSTACK_SECRET_KEY, etc. : variables d'environnement, jamais en DB.
- BYOK (Bring Your Own Key) : clés LLM par organisation, chiffrées en DB (Fernet ou équivalent).
- Mots de passe : bcrypt / argon2.

Scanning

CI backend-security

- pip-audit sur les dépendances (12 CVE fixées dans la session SSRF épinglée).
- Scan automatique à chaque PR.

Runtime

- Rate limiting par IP + par org.
- Détection d'anomalies (optionnel, via Sentry).

Conformité

- RGPD : droit à l'effacement (DELETE /organizations/{id}), export de données.
- SOC 2 (en cours, voir docs/compliance/).
- Data breach notification : api/tasks/compliance.py.

Voir aussi

- docs/admin/RBAC.md -- rôles et permissions
- docs/admin/SSO.md -- configuration SSO
- docs/diagrams/AUTH_FLOW.md -- flux auth
- docs/diagrams/MULTI_TENANCY.md -- isolation tenant

Radiography - executive summary

00 — Synthèse exécutive

Audit en lecture seule du **2026-10-10**, dépôt `rag-saas-platform`, HEAD `59d5ec9` (fusion de la PR n°20 dans `main`). Détails et limites : `15_METHODOLOGIE_ET_LIMITES.md`. Aucune note globale n'est donnée : elle serait artificielle.

En une page

Le dépôt contient une plateforme RAG multi-tenant **très large** : 943 opérations HTTP, 177 tables, 136 migrations, 57 pages frontend, 6 130 tests backend découverts, $\approx 87\,000$ lignes de Python d'API. La qualité de l'**ingénierie de la preuve** est inégale : beaucoup de code et beaucoup de tests, mais la preuve repose surtout sur SQLite et sur des fournisseurs simulés ; **aucune preuve de fonctionnement en production n'a été obtenue** dans cet audit (aucun service distant n'a été appelé).

Ce qui est prouvé (résultat observé, daté)

- Lint, typage et tests frontend : `ruff check api` propre ; `tsc --noEmit`, `eslint .` : code 0 ; `vitest` : **27 fichiers, 178 tests passés** (2026-10-10, exécutés dans cet audit).
- CI distante sur HEAD `59d5ec9` (consultée en lecture) : GitHub Actions **CI success** (5 jobs, dont `backend-tests` et `backend-security`) ; CircleCI `api-tests` **success** (couverture mesurée **77,41 %** des lignes de `api/` sur le sous-ensemble de tests du job) et `rag-pipeline-regression` **success**.
- Corrections de la session du 2026-10-10 (R1–R7, BILL-011/013/014/015/017/020, TEN-003 partiel/005, MAP-003, TEN-012 par l'utilisatrice) : chacune a un test qui échouait avant et passe après ; les verrous (abonnements, crédits) sont prouvés sur une **base PostgreSQL jetable locale**, mais ces tests **ne tournent pas en CI**.
- Machine d'états des factures et règles d'écriture financière réservées au superadmin : testées (mockées), CI verte.

Ce qui est seulement implémenté (code observé, fonctionnement non prouvé)

Pipeline RAG de bout en bout (upload → extraction → chunking → embeddings → recherche hybride → réponse citée), OCR, antivirus (désactivé par défaut), agents autonomes, workflows, MCP/A2A, plugins avec sandbox, voix/Twilio, SSO/WebAuthn, exports RGPD, Stripe/Paystack avec de vrais fournisseurs, tâches Celery périodiques. Voir `06`, `07`, `09`.

Ce qui est incomplet ou contradictoire

- Chunking** avancé non câblé dans le pipeline principal ; chemin de recherche vectorielle (numpy vs HNSW) incertain.
- Pas de remboursement (BILL-016) ; pas d'achat de crédits via Paystack ; devises incohérentes (BILL-018).
- Pages de **demande** de restauration/réactivation absentes de l'interface.
- Documentation contradictoire : `agents.md` (chemins d'API inexistants), `render.yaml` (prototype Streamlit), cahier des charges (515 non reproductible, Partie 15 tronquée).
- 122 des 173 constats historiques n'ont aucune trace de correction (non revérifiés, certains probablement déjà corrigés).

Risques les plus importants (détails et preuves : 12)

1. **Isolation tenant sans barrière en base** (RLS sans FORCE ni policy, rôle contournant) : tout repose sur les filtres applicatifs.
2. **Aucun worker/beat Celery dans l'image de production de l'API** : facturation périodique, purge RGPD et ingestion non garanties.
3. **Migrations manuelles et état de production non prouvé** ; 0136 à appliquer ; sauvegarde/restauration non prouvées.
4. **Facturation jamais confrontée à Stripe/Paystack** ; pas de remboursement ; preuve de paiement manuelle non vérifiée.
5. **Qualité RAG non mesurée** (aucune métrique réelle produite ou lue).

Blocages

Externes (non appelés, donc BLOQUÉ dans cet audit) : base Supabase, Render, Vercel, Redis, Stripe/Paystack sandbox, Resend, LLM. Internes : décisions métier ouvertes (remboursement, crédits Paystack, devises, effet des factures sur les abonnements).

Décisions nécessaires

Isolation tenant (applicative documentée ou RLS contraignant) ; hébergement d'un worker Celery/beat ; politique de remboursement ; fournir des clés de **test** Stripe/Paystack ; quel(s) document(s) font foi pour le périmètre (la Partie 15 d'origine est perdue).

Distance avant commercialisation

Non chiffrable honnêtement en durée. Les P0 de 13_PLAN_DE_TRAVAIL_PRIORISE.md (configuration de production, migrations, sauvegardes, worker, facturation en sandbox, isolation) sont des **préalables** ; les P1 (mesure de qualité RAG, routes sans auth classées, tests PostgreSQL en CI) conditionnent la confiance. Aucune estimation de capacité (documents, utilisateurs, requêtes/s) n'est donnée : **NON MESURÉ**.

Index des livrables

01 inventaire et vérification du chiffre 515 · 02 matrice (150 lignes) · 03 architecture · 04 routes (943) · 05 frontend · 06 RAG · 07 sécurité/tenant · 08 base de données · 09 facturation · 10 tests/CI/déploiement · 11 registre de 180 constats · 12 risques · 13 plan · 14 environnements · 15 méthodologie · 16 preuves.

12 — Dettes, risques et bloquants

Audit du 2026-10-10, HEAD `59d5ec9`. Chaque ligne indique si le constat est **confirmé** (preuve citée), **déduit du code** (risque) ou **une hypothèse**. Aucune note globale artificielle.

A. Risques confirmés (par lecture du code et/ou exécution)

#	Risque	Gravité	Preuve	Statut
R-01	Isolation multi-tenant sans barrière en base : RLS activé sur les tables mais 0 <code>FORCE</code> , 0 <code>CREATE POLICY</code> , rôle applicatif documenté comme contournant. Un seul filtre <code>organization_id</code> oublié dans 101 routeurs / 284 services expose les données d'une autre organisation.	Haute	<code>api/alembic/versions/0098_rls_coverage_gap.py</code> (en-tête), recherche textuelle sur 136 migrations, <code>docs/CAHIER_DES_CHARGES.md</code>	CONFIRMÉ (schéma réel non inspecté) ; exploitation effective : aucune démontrée
R-02	Aucun worker Celery / beat dans l'image de production de l'API : <code>Dockerfile.api</code> lance uniquement <code>gunicorn</code> (l'essai d'un worker dans le même conteneur gratuit de 512 Mo a rendu le service injoignable et a été annulé, commentaire du <code>Dockerfile</code>) ; <code>docker-entrypoint.sh</code> n'est « conservé que pour un hôte plus généreux » ; <code>render.yaml</code> ne déclare aucun worker ; le seul worker planifié est un workflow GitHub par rafales de 7 min. Concernés : ingestion de documents (sauf rattrapage), factures périodiques, allocation de crédits, purges RGPD, rappels.	Haute	<code>Dockerfile.api</code> (CMD et commentaire), <code>docker-entrypoint.sh:4-18</code> , <code>render.yaml</code> , <code>.github/workflows/celery-worker.yml</code>	CONFIGURATION CONFIRMÉE ; ce qui tourne réellement sur Render (service de worker créé à la main ?) NON VÉRIFIÉ
R-03	Migrations non automatisées et état de la base de production non prouvé ; 0136 (table <code>notification_templates</code>) pas encore appliquée — tant qu'elle ne l'est pas, les routes de modèles de notification restent en erreur 500 (TEN-006, déduit : la table n'existe pas sans la migration).	Haute	absence de <code>alembic upgrade</code> dans les fichiers de déploiement ; message de l'utilisatrice ; <code>docs/audit/PRODUCTION_BLOCKERS.md</code> P0-2	CONFIRMÉ (déploiement) / DÉDUIT (500)
R-04	Aucune preuve de sauvegarde ni de restauration (RPO/RTO, rollback)	Haute	PROD-009 ; aucun runbook trouvé dans les fichiers examinés ; <code>docs/audit/BACKUP_AUDIT.md</code> non lu en détail	NON ÉTABLI = pas de preuve
R-05	Facturation jamais validée avec un vrai fournisseur : tous les tests utilisent des événements synthétiques ; tests « live » ignorés ; concurrence PostgreSQL non exécutée en CI ; remboursements non gérés (BILL-016) ; plafonds de dépense non atomiques (BILL-012, non revérifié) ; preuve de	Haute	<code>09_BILLING_PAYMENTS.md</code> , tests ignorés (8 live)	CONFIRMÉ

#	Risque	Gravité	Preuve	Statut
	paiement manuelle non confrontée à une banque.			
R-06	Qualité RAG non mesurée : les métriques existent dans le code (Recall@K, MRR, NDCG) mais aucune valeur réelle n'a été produite ou lue ; chemin de recherche de production incertain (numpy vs HNSW) ; chunking avancé non câblé	Moyenne-haute	06_RAG_PIPELINE.md	CONFIRMÉ (absence de mesure)
R-07	Documentation contradictoire/obsolète : agents.md (endpoints /api/eval/* et /mcp/v1/servers... inexistants, Python 3.11, dossier api/eval/ absent) ; render.yaml ne décrit que le prototype Streamlit ; cahier des charges (515 non reproductible, texte d'origine perdu, Partie 15 tronquée, note « GitHub Actions désactivé » périmée). Les modes Bob (FACTORY/GUARDIAN/AUTOPSY/CHANGELAB) reposent sur ces chemins.	Moyenne	06, 01, 15	CONFIRMÉ
R-08	Couverture et fiabilité de la preuve : tests principalement sur SQLite ; 183 des 943 opérations HTTP n'ont aucun test référençant leur chemin (heuristique) ; 84 opérations sans dépendance d'authentification détectée (à classer : publiques légitimes ou protégées autrement)	Moyenne	04_API_ENDPOINTS.csv, 08	CONFIRMÉ (heuristique)
R-09	Rate limiting dégradé sans Redis : repli sur un limiteur en mémoire de processus (non partagé entre workers, remis à zéro à chaque redémarrage)	Moyenne	journaux de tests (« degrading to the in-process limiter »), api/security/rate_limit.py	CONFIRMÉ
R-10	/metrics public si METRICS_AUTH_TOKEN est absent	Moyenne	api/main.py:426-444	CONFIRMÉ (code) ; valeur en production NON VÉRIFIÉE

B. Risques déduits du code (non mesurés)

- Mémoire : torch + sentence-transformers + unicorn (4 workers par défaut) + worker Celery dans 512 Mo ; démarrage à froid (PROD-004) ; plan gratuit Render.
- Chargement en mémoire de tous les chunks d'une organisation (BM25 / repli numpy) : dégradation avec la taille des organisations.
- Dispatch Celery en « best effort » : un document peut rester pending si le broker est indisponible (un rattrapage existe, RAG-003, à vérifier en production).
- Importeurs externes asynchrones (URL, GitHub, Drive, Notion...) appellent encore boto3 de façon synchrone (TEN-003 partiel).
- api/tools/sql_tool.py : SQL dynamique via text() ; calculator.py : eval() borné par AST.
- Clés Redis et caches : prise en compte du tenant non établie.
- 122 constats historiques sans trace de correction (voir 11) : certains sont probablement déjà corrigés sans mention d'identifiant.

C. Hypothèses nécessitant une mesure

Capacité (documents / utilisateurs / req/s) : **NON MESURÉ** ; latence du chat ; coût LLM par organisation ; comportement sous 8 inscriptions concurrentes réelles ; efficacité de la détection d'injection.

D. Dettes techniques

- Deux pipelines RAG (`src/` Streamlit et `api/`), deux CI (GitHub, CircleCI) dont la durée approche la limite (CircleCI $\approx$ 55 min pour une limite de 60 ; GitHub $\approx$ 42-60 min pour 75) ; liste de fichiers de test CircleCI codée en dur (nouveaux tests non couverts).
- Fichiers volumineux : `ROADMAP.md` 300 Ko, cahier des charges 649 Ko ; `snyk.exe` (55 Mo) présent à la racine mais **ignoré par Git** (`.gitignore:25` , non suivi).
- Dépôt local encombré : 19 branches locales, 10 worktrees, répertoires de travail d'agents (`.claude/`).
- Cinq suites de test du prototype exclues de la CI.

E. Bloquants avant mise en production commerciale

1. R-01 (décision : accepter l'isolation applicative documentée, ou introduire des politiques RLS + rôle non contournant).
2. R-02 / R-03 / R-04 (exploitation : worker, migrations, sauvegardes).
3. R-05 (validation sandbox Stripe/Paystack de bout en bout, politique de remboursement).
4. Une mesure de qualité RAG sur un jeu réel (R-06).

F. Dépendances externes bloquantes (BLOQUÉ — SERVICE EXTERNE dans cet audit)

Stripe/Paystack (sandbox), Supabase (schéma réel), Render/Vercel (état réel), Redis réel, Resend, fournisseurs LLM, ClamAV. Aucun n'a été appelé.

G. Décisions métier encore ouvertes (relevées pendant le travail du 2026-10-10)

Politique de remboursement (`refunded` non géré) ; effet d'une facture payée/annulée sur le statut d'abonnement (aujourd'hui aucun, documenté) ; achat de crédits via Paystack ; devise des packs (USD) vs plans (EUR) ; V6 (horodatage Paystack) et V10 (noms des clés de permission) : « ne rien changer » à ce jour ; rôle du viewer sur les workflows (décidé et corrigé le 2026-10-10).